

CDS View 課程練習 6 : CDS Access Control

Lecture

這一課要教什麼

cds01 ~ cds05 建的每個 CDS View，`@AccessControl.authorizationCheck` 都寫 `#NOT_REQUIRED`——意思是「完全不做權限檢查，誰都能查全部資料」。這一課要換成真正的權限控管：**CDS Access Control**，用一種宣告式語言（CDS DCL，Data Control Language）定義「誰能看到哪些資料列」，這個限制會透明地套用在任何消費這個 View 的地方（Open SQL、RAP、Fiori Elements），不需要每支呼叫端程式自己記得加 `WHERE` 條件。

語法元素講解

① `@AccessControl.authorizationCheck` 的三個合法值：

值	意思
<code>#NOT_REQUIRED</code>	完全不做權限檢查（cds01 ~ cds05 都用這個）
<code>#CHECK</code>	一定要有對應的 CDS Role（DCL）定義存取規則，沒有規則就等於沒有人能存取任何資料
<code>#PRIVILEGED_ONLY</code>	完全不受權限檢查限制，但只有「被明確標記為特權」的呼叫端（例如另一個標了 <code>@AccessControl.privilegedOnly: true</code> 的 View）才能繞過去查——一般消費端（Open SQL 直接查）依然會被擋下來。這一課不會實際建這種物件，先知道有這個選項即可，實務上比較少用（通常只在框架內部、其他限制已經做過檢查的場景使用）。

② `define role`（CDS DCL）語法：另外建一個獨立物件（型別 `DCLS/DL`，跟 CDS View 的 `DDL/DF` 是不同型別），內容用 `grant select on <被保護的 CDS Entity> where <條件>;`：

```
define role <角色名稱> {
  grant select on <CDS Entity>
    where <element> <比較運算子> <值>;
}
```

⚠ 這系統實測出來的一個必要限制：這個 DCL Role 一定要加 `@MappingRole: true` annotation，缺了會在啟用時報：

```
DCLs without annotation "@MappingRole: true" are not supported
```

這是照抄這系統既有標準物件（`I_CAPaymentOrder` 的 DCL Role）驗證出來的正確寫法，不是猜測——這個系統上所有正常運作的標準 DCL Role 都帶著這個 annotation。

③ `WHERE` 條件支援的比較類型（官方文件 `ABENCDS_F1_COND_LITERAL`）：

- 字面值比較：`carrid = 'AA'` (這一課用的範例)
- **Session Variable 比較**：可以用 `$session.user / $session.system_date / $session.user_date / $session.user_timezone / $session.system_language` (注意這個清單沒有 `$session.client`——因為 Client 隔離本來就是自動處理的，不需要另外寫規則)
- **PFCG Authorization Object 比較** (`aspect pfcg_auth(...)`)：串接傳統 PFCG 角色的權限物件——這門課不會實際建這種規則，因為這個系統的權限物件 (Authorization Object，SU21 維護) 完全沒有 ADT API (本課程沿用的另一門課程已經查證過這個限制)，只能請你自己在 SU21 手動建好權限物件後，回頭在 DCL 裡引用它的名稱。

這一課用最單純的字面值比較示範核心機制，讓你先搞懂「Access Control 怎麼運作」，不糾結在 PFCG 整合這個需要額外 GUI 操作的部分。

完整範例

Layer 1 : `ZI_CDS06_FLIGHT` (#CHECK)

```
@AbapCatalog.sqlViewName: 'ZICDS06FLGT'
@AccessControl.authorizationCheck: #CHECK
@endUserText.label: 'CDS06: Flight with Access Control (#CHECK)'
define view ZI_CDS06_FLIGHT
  as select from sflight
{
  key carrid,
  key connid,
  key fldate,
  price,
  currency,
  seatsmax,
  seatsocc
}
```

Layer 2 : `ZI_CDS06_FLIGHT` (DCL Role)，物件名稱刻意跟 CDS View 同名——這是官方標準物件的命名慣例，兩者是不同物件型別所以不會衝突)

```
@endUserText.label: 'CDS06: Access rule - only AA carrier visible'
@MappingRole: true
define role ZI_CDS06_FLIGHT {
  grant select on ZI_CDS06_FLIGHT
  where carrid = 'AA';
}
```

這個規則的意思是：任何人查詢 `ZI_CDS06_FLIGHT`，不管查詢語句本身有沒有寫 `WHERE` 條件，系統都會自動只回傳 `carrid = 'AA'` 的資料列——這不是呼叫端自己加的篩選，是 CDS Access Control 在資料庫層透明套用的隱性限制。

⚠️ 核心觀念：Access Control 不是「呼叫端自己記得加條件」，是系統強制套用的隱性篩選

這一課的驗證程式故意寫了一句完全沒有 **WHERE** 條件、看起來會查到所有航空公司的 Open SQL：

```
SELECT carrid, connid, fldate
FROM zi_cds06_flight
INTO TABLE @DATA(lt_checked).
```

實際執行結果只回傳了 25 筆，而且全部都是 `carrid = 'AA'`——證實 DCL Role 的限制**不需要呼叫端配合**，系統會自動在背後套用。對照組 `ZI_CDS02_FLIGHT` (`#NOT_REQUIRED`，cds02 建的) 用同樣「沒寫 WHERE」的查詢方式，正常查到 356 筆、橫跨 8 家不同航空公司——這就是有沒有 Access Control 的具體差異。

這正是 **Access Control** 相對「呼叫端自己寫 **WHERE** 條件」的核心價值：只要有一支程式忘記加篩選條件 (不管是無意的疏忽，還是惡意繞過)，沒有 Access Control 的 View 就會洩漏不該給這個使用者看的資料；有 Access Control 的話，系統會強制套用，呼叫端寫不寫篩選條件都不影響安全性。

跟 **AUTHORITY-CHECK** 的分工

AUTHORITY-CHECK 是傳統的 ABAP 語句，寫在程式碼裡明確檢查某個權限物件 (Authorization Object，SU21 維護)：

```
AUTHORITY-CHECK OBJECT 'S_TCODE'
  ID 'TCD' FIELD 'SE38'.
IF sy-subrc <> 0.
  " 沒有權限，處理例外
ENDIF.
```

兩者的分工原則：

	CDS Access Control (DCL)	AUTHORITY-CHECK
套 用 方 式	宣告式，自動套用在 任何 消費這個 CDS Entity 的地方 (Open SQL、RAP EML、Fiori Elements)	命令式，只在 你明確寫了這行程式碼 的地方才會檢查
適 用 場 景	「查詢結果要不要包含某些資料列」這種 資料層級 的過濾	「這個使用者能不能執行某個動作」這種 操作層級 的判斷 (例如能不能按下某個按鈕、能不能執行某個交易)，或是需要複雜自訂邏輯、不是單純資料列過濾的情境
遺 漏 風 險	只要 CDS Entity 標了 <code>#CHECK</code> ，不會有人「忘記檢查」	呼叫端寫程式時如果忘記加這行，權限檢查就完全不會發生

實務上兩者常常搭配使用：CDS Access Control 顧資料查詢這一層，**AUTHORITY-CHECK** 顧「這個操作本身准不准做」這一層——這一課只示範 CDS Access Control 的部分 (因為完整可以在這個系統上建立、驗證)，**AUTHORITY-CHECK** 的語法列出來給你對照，沒有另外建一個自訂權限物件來執行它 (因為這系統的權限物件是 GUI-only，不在這門課的驗證範圍內)。

Eclipse ADT 建立 CDS View 與 DCL Role : Step by Step

1. 建 `ZI_CDS06_FLIGHT` : 對著 `$TMP` 套件右鍵 → New → Other ABAP Repository Object → Data Definition → Name `ZI_CDS06_FLIGHT` → Templates 選 **Define View (obsolete as of AS ABAP 7.57)** → Reference Object 選 `SFLIGHT` → 改成上面的內容 → Ctrl+S → Activate
2. 建 DCL Role : 對著 `$TMP` 套件右鍵 → New → Other ABAP Repository Object → 篩選 **Access Control** (DCL Source) → Name 填 `ZI_CDS06_FLIGHT` (可以跟 CDS View 同名, 因為型別不同) → 精靈通常會自動帶出一段骨架, 記得檢查有沒有 `@MappingRole: true` 這行, 沒有的話手動加上
3. 改成上面 DCL Role 的完整內容 → Ctrl+S → Activate
4. 注意順序: CDS View 要先啟用成功, DCL Role 才能正確解析 `grant select on <CDS View 名稱>`
5. 對 `ZI_CDS06_FLIGHT` 用 **Data Preview**, 確認只查得到 `carrid = 'AA'` 的資料 (其他航空公司的資料完全看不到)

這一課學到的東西, 接下來會怎麼用

- cds07: 聚合——這一課的 Access Control 概念會延續到聚合查詢 (聚合前的明細資料一樣受 Access Control 限制)
- cds08 (期中整合) : 把 Access Control 疊進最終的多層 View 案例
- 進階篇: RAP 課程 (如果你上過) 的 BDEF `authorization` 子句, 是 RAP 世界另一套權限機制, 跟這一課教的 CDS Access Control 是不同層次 (DCL 顧的是「查詢」, BDEF `authorization` 顧的是「CUD 操作」), 但概念上是同一個「宣告式、系統自動套用」的設計哲學

Eclipse ADT Step by Step (重點回顧)

1. `ZI_CDS06_FLIGHT` (CDS View, `#CHECK`) : 以 `SFLIGHT` 為來源
2. `ZI_CDS06_FLIGHT` (DCL Role, 型別 **Access Control**) : `@MappingRole: true + grant select on ZI_CDS06_FLIGHT where carrid = 'AA';`
3. CDS View 先啟用, DCL Role 才能啟用
4. Data Preview 驗證只看得到 `AA` 的資料

學習目標

- 能講出 `@AccessControl.authorizationCheck` 三個合法值 (`#CHECK`/`#NOT_REQUIRED`/`#PRIVILEGED_ONLY`) 的差異
- 能寫出一個基本的 `define role` (CDS DCL) , 包含這系統實測要求的 `@MappingRole: true` annotation
- 能講出 CDS Access Control 支援的三種條件類型 (字面值、Session Variable、PFCG Authorization Object) , 並知道這一課示範哪一種、為什麼另外兩種在這個系統上比較難完整示範
- 能講出「Access Control 是系統強制套用的隱性篩選, 不是呼叫端自己加的 WHERE 條件」這個核心觀念, 並能舉出這一課的實測證據 (沒寫 WHERE 條件的查詢依然被自動篩選)
- 能講出 CDS Access Control 跟 `AUTHORITY-CHECK` 的分工原則 (資料層級 vs. 操作層級)

物件清單

物件	名稱	型別
CDS Interface View (<code>#CHECK</code>)	<code>ZI_CDS06_FLIGHT</code>	DDLS/DF

物件	名稱	型別
CDS DCL Role	ZI_CDS06_FLIGHT	DCLS/DL
驗證程式	ZR_CDS06_DEMO	PROG/P

全部物件都在 **\$TMP** 套件，已建立並啟用（讀取 active 版本內容與寫入內容逐字相符）。

動手練習物件（你自己動手建，名稱自訂，不算在上面正式的課程物件清單裡）：

物件	建議名稱	型別	對應練習
CDS View + DCL Role (用 Session Variable 條件)	ZI_CDS06_MY_FLIGHT (自訂)	DDLS/DF + DCLS/DL	動手練習

動手練習

輪到你了：建一個新的 CDS View + DCL Role 組合（物件名稱、套件 **\$TMP**，命名自訂），要求：

1. CDS View 基於 **SFLIGHT**（或沿用 **ZI_CDS06_FLIGHT** 也可以，但這一題鼓勵你自己重新建一次加深印象），**@AccessControl.authorizationCheck: #CHECK**
2. DCL Role 用 **Session Variable** 當條件，而不是字面值——例如 **where currency = 'USD'**（不算 session variable，這裡示範一下也可以）或更貼近 Session Variable 的練習：試著寫一個永遠為真 / 永遠為假的 **\$session.system_date** 條件（例如 **where fldate <= \$session.system_date**，只顯示「今天以前」的航班），觀察結果
3. 別忘了 **@MappingRole: true**
4. 用 Data Preview 驗證：確認查詢結果真的只有符合條件的資料列

建好、啟用成功後跟我說一聲（貼程式碼或截圖都可以），我會幫你核對語法。

如果你想額外挑戰一下：試著故意把 DCL Role 的 **@MappingRole: true** 拿掉，重新啟用一次，實際看一次這系統的錯誤訊息（**DCLs without annotation "@MappingRole: true" are not supported**）——親眼看過這個錯誤，比只是讀講義印象更深。

驗證方式

ZR_CDS06_DEMO 透過 **programrun** 無頭驗證，用「完全沒寫 **WHERE** 條件」的查詢方式分別查 **#CHECK**（有 DCL Role）跟 **#NOT_REQUIRED**（無限制）兩個 View，確認前者被自動篩選、後者不受影響：

```
=== 1. ZI_CDS06_FLIGHT (#CHECK, DCL role restricts to carrid = AA) ===
Query intentionally asks for ALL carriers (no WHERE clause):
AA 0017 2018/10/29
AA 0017 2018/11/30
AA 0017 2019/01/01
AA 0017 2019/02/02
AA 0017 2019/03/06
Total rows returned:          25
=== 2. Distinct carriers actually returned (should be AA only) ===
```

```
MATCH: DCL access control silently filtered to carrid = AA, row count      25
=== 3. ZI_CDS02_FLIGHT (#NOT_REQUIRED, no access control) ===
    Same kind of query, no WHERE clause:
Total rows returned:      356 | distinct carriers:      8
MATCH: without access control, multiple carriers are visible (no silent filtering)
```

`ZI_CDS06_FLIGHT` 的查詢完全沒有寫 `WHERE`，卻只回傳 25 筆、全部都是 `AA`——證實 DCL Role 的篩選是系統自動套用的，不需要呼叫端配合；對照組 `ZI_CDS02_FLIGHT` 同樣沒寫 `WHERE`，正常回傳 356 筆、橫跨 8 家航空公司，證實只有標了 `#CHECK` 且有對應 DCL Role 的 View 才會有這個隱性篩選行為。

動手練習的驗證方式：Eclipse 啟用成功 + 用 Data Preview 看查詢結果，貼程式碼給我核對語法。

思考題

1. 如果 `ZI_CDS06_FLIGHT` 標了 `#CHECK`，但沒有對應的 DCL Role（或 DCL Role 存在但沒有啟用），你覺得查詢這個 View 會發生什麼事？（提示：回顧 `#CHECK` 的定義——「一定要有規則」）
2. 這一課的 DCL Role 用字面值 `carrid = 'AA'` 寫死，代表所有使用者查這個 View 都只看得到 `AA` 的資料，沒有「依不同使用者顯示不同資料」的效果。如果想要「業務員只看得到自己負責的航空公司」，你覺得 DCL 條件該怎麼改？（提示：回顧 PFCG Authorization Object 條件、或 Session Variable `$session.user` 條件）
3. `cds02` 的 `ZI_CDS02_FLIGHT`（`#NOT_REQUIRED`）跟這一課的 `ZI_CDS06_FLIGHT`（`#CHECK`）底層都是查 `SFLIGHT`，如果現在有一支新程式想查詢航班資料，你會建議用哪一個 View？在什麼情境下該用哪一個？
4. 如果同一個 CDS Entity 需要「大部分情境下都要做權限檢查，但某個特定內部框架流程需要繞過檢查」，你會怎麼設計？（提示：回顧 `#PRIVILEGED_ONLY` 的說明）

答案

見 `zi_cds06_flight.ddls.abap`、`zi_cds06_flight.dcls.abap`、`zr_cds06_demo.prog.abap`。SAP 端物件：`ZI_CDS06_FLIGHT`（CDS View）、`ZI_CDS06_FLIGHT`（DCL Role，型別不同不衝突）、`ZR_CDS06_DEMO`（驗證程式）。動手練習（Session Variable 條件的 CDS View + DCL Role）由你在 Eclipse 動手建立，沒有固定答案快照——建好後跟我核對即可。